

Class starts at 9:05 PM

Agenda

- 1) Custom validators for LLM output
- 2) Decorators & Validation Boundaries
- 3) Why container?
- 4) Using Docker
- 5) Demo (AI application)

Custom Validation.

We will pass something back to you as feedback loop

5

Cust

Go beyond basic types – add *business rules and cross-field logic*.



Validate Data.
Enforce Rules.
Build Trust.



Why Custom Validation?

Built-in types check structure.
Custom validators check meaning.
Use them to enforce business rules and data consistency.

- 
- ### Types of Validators

 - **Field Validators**
Validate a single field.
 - **Model Validators**
Validate across multiple fields or the whole model.

- 
- ### Validator Modes

 - **before**: run before standard validation (raw input)
 - **after**: run after standard validation (parsed value)
 - **wrap**: get full control (around validation)

Example: Field & Model Validators

```
from pydantic import BaseModel, field_validator, model_validator, ValidationError
from typing import Literal

class UserCreate(BaseModel):
    name: str
    email: str
    age: int
    role: Literal["user", "admin"] = "user"
    confidence: float # between 0 and 1

# 1. Field Validator (after)
@field_validator('email')
@classmethod
def validate_email(cls, v: str) -> str:
    if "@" not in v:
        raise ValueError("Invalid email address")
    return v.lower()

# 2. Field Validator (before)
@field_validator('age', mode='before')
@classmethod
def coerce_age(cls, v):
    if isinstance(v, str):
        return int(v) # e.g., "25" -> 25
    return v

# 3. Model Validator (after)
@model_validator(mode='after')
def check_business_rules(self):
    if self.role == "admin" and self.confidence < 0.8:
        raise ValueError("Admin confidence must be >= 0.8")
    if self.age < 18:
        raise ValueError("User must be at least 18 years old")
    return self
```

Validation in Action

**Valid Input**

```
UserCreate(
    name="Alice",
    email="Alice@example.com",
    age=25,
    role="admin",
    confidence=0.95
)
```

✓ Model created successfully

**Invalid: Bad Email**

```
UserCreate(
    name="Bob",
    email="bobexample.com", # missing @
    confidence=0.9
)
```

ValidationError: Invalid email address

**Invalid: Business Rule**

```
UserCreate(
    name="Carol",
    email="carol@example.com",
    age=30,
    role="admin",
    confidence=0.3 # too low
)
```

ValidationError: Admin confidence must be >= 0.8

- Common Use Cases
- **Range & Threshold Checks**
e.g., confidence between 0 and 1
- **Allowed Values**
e.g., role must be from a fixed list
- **Cross-Field Rules**
e.g., if role is admin, confidence must be high
- **Format & Pattern Checks**
e.g., email, phone, IDs (regex)
- **Conditional Requirements**
e.g., if is_active, then email_verified must be True



Pro Tip

Keep validators small, focused, and test them!
They are executed every time data enters your system.

① You define User

```
class User (base Model):  
    name: str  
    age: int
```

② SDK imports the class locally

③ SDK sends the HTTP request
↳ This request will travel to OpenAI

④ Open AI model will generate json

⑤ SDK receive the response

⑥ user = response.output_parsed

mode = wrap

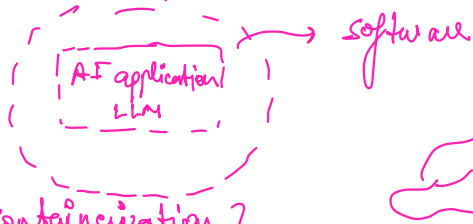
it receives a handler that will perform your normal
validation

↓ - Raw data
- before validation

wrap validator ←

Hydrate performs validation
after validation
final

why this topic is important? $\rightarrow$ A.S



why containerization?

Docker will wrap our application into something called as container



isolated



library dependency, versioning dependency, installed package



7 Decorators & Validation Boundaries

Wrap functions with *validation*, *logging*, and *safeguards*.



Validate In.
Validate Out.
Trust the Boundary.



Why Use Decorators?

- Enforce validation at function boundaries
- Keep business logic clean
- Reusable, consistent, and composable
- Add logging, timing, and error handling
- Essential for production-grade AI systems



Common Responsibilities

- Validate inputs (type + schema)
- Validate outputs (type + schema)
- Log requests, responses, and errors
- Measure execution time
- Handle exceptions gracefully



Key Benefits

- Consistency**
Same checks everywhere
- Reusability**
One decorator, many functions
- Safety**
Bad data never enters the core
- Observability**
Logs, errors & metrics built-in

Example: Validation Decorator

```
from functools import wraps
from pydantic import BaseModel, ValidationError
from time import perf_counter
from typing import Callable, Type

def validate_in(
    input_model: Type[BaseModel],
    output_model: Type[BaseModel]
) -> Callable:
    def decorator(func: Callable) -> Callable:
        @wraps(func)
        def wrapper(data: dict) -> dict:
            start = perf_counter()

            # 1. Validate Input
            try:
                validated_input = input_model.model_validate(data)
            except ValidationError as e:
                raise ValueError(f"Invalid input: {e}")

            # 2. Execute Function
            result = func(validated_input)

            # 3. Validate Output
            try:
                validated_output = output_model.model_validate(result)
            except ValidationError as e:
                raise ValueError(f"Invalid output: {e}")

            # 4. Logging & Timing
            duration = (perf_counter() - start) * 1000
            print(f"[{func.__name__}] executed in {duration:2f} ms")
            return validated_output.model_dump()

        return wrapper

    return decorator
```

How It Works

Input Data

Raw dict / JSON

Validate Input

Pydantic Input Model

Execute Function

Business Logic

Validate Output

Pydantic Output Model

Log & Return

Safe, Typed Output

Example Usage

```
class UserIn(BaseModel):
    user_id: int
    email: str
    age: int

class UserOut(BaseModel):
    user_id: int
    email: str
    status: str

@validate_in(UserIn, UserOut)
def create_user(user: UserIn) -> dict:
    # Business logic here
    return {
        "user_id": user.user_id,
        "email": user.email,
        "status": "created"
    }

# Call with raw data
response = create_user({
    "user_id": 123,
    "email": "andreas@sample.com",
    "age": 29
})
```

Best Practices

- Keep decorators small and focused
- Use specific Pydantic models for clarity
- Log enough context (but avoid sensitive data)
- Fail fast on validation errors
- Compose decorators when needed



Key Takeaway

Decorators let you build strong validation boundaries around your functions—ensuring only **clean, typed** data reaches your core logic.



Stronger
Data Integrity



Better
Observability



Clean
Business Logic



Production
Ready

MODULE 1

Why Containerization?

Consistent, portable and reliable environments are critical for AI applications.



The Problems We Face



The "Works on My Machine" Problem
Code runs on your machine but fails in staging or production due to environment differences.



Environment Drift
Over time, systems change, dependencies update, and setups become inconsistent.



Why AI Applications Are Harder to Deploy
AI apps have heavy and complex dependencies, making reproducibility challenging.



Native Dependencies
Libraries like NumPy, PyTorch, FAISS, OpenCV require system-level dependencies.



Python Package Conflicts
Different projects need different versions of packages, leading to conflicts.



GPU / CPU Compatibility
AI frameworks depend on specific CUDA, cuDNN, or hardware combinations.



Reproducibility
Hard to reproduce results or deploy reliably without a consistent environment.

Before vs After Containerization

BEFORE Traditional Deployment

- ✗ Inconsistent environments
- ✗ Long setup & onboarding time
- ✗ Dependency conflicts
- ✗ Hard to debug environment issues
- ✗ Not portable
- ✗ Difficult to scale reliably
- ✗ Low reproducibility

AFTER With Containerization

- ✓ Consistent environment everywhere
- ✓ Fast setup & easy onboarding
- ✓ Isolated dependencies
- ✓ Easier debugging & troubleshooting
- ✓ Portable across systems & clouds
- ✓ Easier to scale
- ✓ High reproducibility



Goal

Containerization solves these challenges by packaging your AI application and its environment together—ensuring it runs the same way everywhere.

What helps in doing containerization?

Docker is a software that helps you doing this containerization

1) Docker Images → stored like a repository

2) Containers

3) Registries

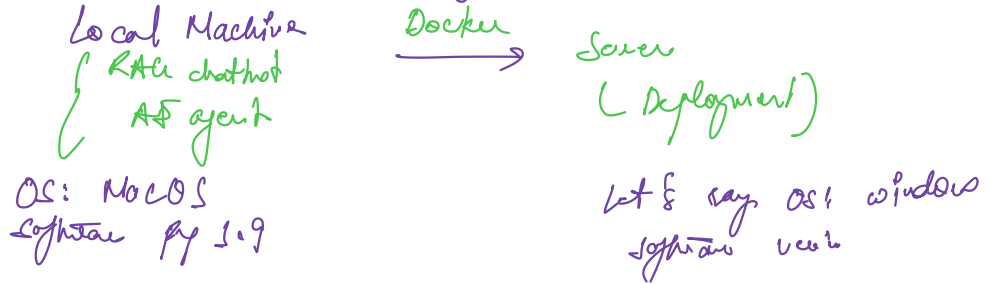
Something like github where you can put your code
+ other people can also put this code and that strc
is known as docker hub

What do we store in docker hub?

→ store docker image

In AI application?

When I need to do any deployments that is where I need docker. Not required for POC



RAL application

Does RAL have multiple components?

Fine-tuning vs RAL

My org data
keeps updating
RAL

PT $\rightarrow$ Training happens?

I want to create PT
my own llm for enterprise.

User



US



Postmark (Font API)



this will help me in tapping
my main.py function



ark question.py



rag.py

This rag.py will get data from two places

vector → opens 12114

context

↓
response will be generated

→ we will be building two small rag applications & then we will containerize it

Old docs $\rightarrow$ chunk + hash

New doc $\rightarrow$ chunk + hash

Compare hashes

Same

$\downarrow$



Re-embed

no need
of embedding
again

Hah planned
Employees may request a leave before 7 days

→ [8fabcdre] ←

↑ ↑ ↑

Same test → same hah

Different test → different hah

refund	within	7 days	→	WXYZ2
"	"	30 days	→	ABCD
"	"	30 days	→	ABCD

MODULE 2

Docker Fundamentals

Docker lets you package applications and their dependencies into portable, consistent units called containers.

Key Concepts

**Images**

Read-only templates used to create containers. They include the app code, runtime, libraries and dependencies.

**Containers**

Runnable instances of images. They are isolated, lightweight and portable.

**Registries**

Repositories to store and share Docker images. Example: Docker Hub, Private Registry.

**Image vs Container**

An image is a blueprint; a container is a running instance of that blueprint.

Image vs Container	
Image (Template)  • Read-only • Built from Dockerfile • Stored in registry	Container (Running Instance)  • Read-write layer • Isolated process • Can be started, stopped, deleted

**Why it matters?**

Docker provides consistency across environments, faster setup for AI/ML applications where dependencies and environments are critical.

MODULE 3

Building an AI / RAG Application

In this module, we build a complete Retrieval-Augmented Generation (RAG) application from scratch that we will containerize in the upcoming modules.



What We Are Building



A RAG system that can answer user questions using our own documents.

Why RAG?

- ✓ Grounds answers in your data
- ✓ Reduces hallucinations
- ✓ Keeps knowledge up to date
- ✓ Improves transparency and trust

High Level Architecture



Components We Build

Component	Description	Key File / Tech
1. Knowledge Base Documents	Collection of domain documents that the system will use to answer questions.	/docs/
2. Document Ingestion	Loads documents and splits them into chunks for processing.	ingest.py (load & split)
3. Embedding Generation	Converts text chunks into vector embeddings using an embedding model.	OpenAI Embeddings
4. Vector Database	Stores embeddings for similarity search and retrieval.	ChromaDB
5. Retrieval	Given a user question, retrieves the most relevant document chunks from the vector DB.	Similarity Search
6. Generation (Answer)	Sends retrieved context + question to the LLM to generate a grounded answer.	LLM (Chat) + Prompt
7. API Server	Exposes an HTTP endpoint so users can ask questions via an API.	api.py (FastAPI)

Tech Stack

- Python
- FastAPI
- OpenAI (Embeddings / LLM)
- ChromaDB (Vector DB)
- PyPDF2 / DOCX (Text Loaders)
- tiktoken (Text Splitter)

Key Takeaway



By the end of this module, we have a working RAG application with ingestion, retrieval and a question-answer API — ready to be containerized with Docker!

Files Created in This Module

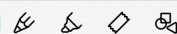


- docs/
- ingest.py
- api.py
- requirements.txt
- .dockerignore



Next Up

In the next modules, we will containerize this application using Docker and make it production-ready!



Cust        

Go beyond basic types – add *business rules* and *cross-field logic*.



**Validate Data.
Enforce Rules.
Build Trust.**



Built-in types check structure.
Custom validators check meaning.
Use them to enforce business
rules and data consistency.



- **Field Validators**
Validate a single field.
- **Model Validators**
Validate across multiple fields or the whole model.



- **before**: run before standard validation (raw input)
- **after**: run after standard validation (parsed value)
- **wrap**: get full control (around validation)

```
from pydantic import BaseModel, field_validator, model_validator, ValidationError
from typing import Literal

class UserCreate(BaseModel):
    name: str
    email: str
    age: int
    role: Literal["user", "admin"] = "user"
    confidence: float # between 0 and 1

# 1. Field Validator (after)
@field_validator('email')
@classmethod
def validate_email(cls, v: str) -> str:
    if "@" not in v:
        raise ValueError("Invalid email address")
    return v.lower()

# 2. Field Validator (before)
@field_validator('age', mode='before')
@classmethod
def coerce_age(cls, v):
    if isinstance(v, str):
        return int(v) # e.g., "25" -> 25
    return v

# 3. Model Validator (after)
@model_validator(mode='after')
def check_business_rules(self):
    if self.role == "admin" and self.confidence < 0.8:
        raise ValueError("Admin confidence must be >= 0.8")
    if self.age < 18:
        raise ValueError("User must be at least 18 years old")
    return self
```

- **Range & Threshold Checks**
e.g., confidence between 0 and 1
- **Allowed Values**
e.g., role must be from a fixed list
- **Cross-Field Rules**
e.g., if role is admin, confidence must be high
- **Format & Pattern Checks**
e.g., email, phone, IDs (regex)
- **Conditional Requirements**
e.g., if is_active, then email_verified must be True

✔ Valid Input <pre>UserCreate(name="Alice", email="Alice@example.com", age=25, # coerced role="admin", confidence=0.95)</pre> ✔ Model created successfully	✗ Invalid: Bad Email <pre>UserCreate(name="Bob", email="bobexample.com", # missing @ age=28, confidence=0.9)</pre> ValidationError: Invalid email address	✗ Invalid: Business Rule <pre>UserCreate(name="Carol", email="carol@example.com", age=30, role="admin", confidence=0.3 # too low)</pre> ValidationError: Admin confidence must be >= 0.8
--	---	---



Custom validation lets you enforce business rules, ensure data consistency, and prevent bad data from entering your system.



Stronger Data Integrity



Safer AI Outputs & Tool Calls



More Reliable Applications